Kriging with Fuzzy Domain Boundaries

The Idea that was shelved for 14 years and started 20 years ago

Matthew Nimmo

2026-09-03

Abstract In 2012, standard resource estimation offered only two choices at domain boundaries: hard cut-offs (zero samples outside) or soft buffers (all samples inside treated as 100% equal). When I proposed scaling sample weights continuously based on domain membership, the traditional consensus was sceptical. Fourteen years later, with the rise of non-stationary spatial kernels, inverse-variance penalties, and machine-learning spatial proxies, the mathematics has caught up. Here is the formal proof why discrete fuzzy domain membership works.

Table of contents

The challenge

Spatial estimation in geostatistics relies heavily on stationary domains to model spatial continuity accurately. However, real-world geological contacts rarely fit neatly into binary categories:

- Hard Boundaries: Cut off sample selection abruptly at domain wireframes. While this prevents high-grade dilution, it induces a truncation selection bias (Berkson's paradox). This artificially collapses sample variance inside the domain, destroys spatial range, and creates steep, non-physical estimation steps at the contact.
- Soft Boundaries: Include samples across domain boundaries within a fixed buffer zone. While this models smooth physical transitions, standard Ordinary Kriging assigns these outside samples full weight as if they belonged entirely to the estimation domain. This over-smooths sharp geological contacts and introduces artificial dilution.

The challenge is to model continuous or uncertain domain transitions dynamically, treating boundaries as fuzzy, without manually tuning arbitrary buffer zones or diluting the estimation neighbourhood with unweighted outside samples.

The solution

Fuzzy Domain Kriging (FDK) modifies the linear system equations to damp the spatial covariance or weight contribution of samples based on their domain membership $u_i \in [0, 1]$, where $u_i = 1$ indicates complete domain membership and $u_i = 0$ indicates total exclusion.

Three distinct mathematical variants offer different operational trade-offs:

- **Variant A** - Multiplicative covariance scaling ($w = k(D \cdot K)^{-1}$): Modifies the cross-covariance matrix K directly via an interaction mask matrix D, dampening spatial correlations between samples across fuzzy domain boundaries. Use Case: Localised domain boundary estimation where off-diagonal spatial correlations across complex, non-stationary contacts must be explicitly modulated.
- **Variant B** - Additive variance penalty ($w = k(K + R)^{-1}$): Introduces a diagonal observation error penalty matrix R, where low membership $u_i \to 0$ acts as an infinite variance penalty $R_{ii} \to \infty$. Use Case: General-purpose resource estimation across soft boundaries, guaranteeing positive definiteness and matrix invertibility under all sample configurations.
- **Variant C** - Post-projection weight scaling ($w = w_{fixed}M$): Pre-computes unconditioned master weights $w_{fixed} = kK^{-1}$ on a regular grid and post-scales them using a diagonal fuzzy operator matrix M. Use Case: Ultra-fast Sequential Gaussian Simulation (SGS). Bypasses $O(N^3)$ matrix inversions at every random-path simulation node, reducing operations to $O(N)$ vector scaling.

Theoretical foundations and structural framing

Central premise

Premise 1 - Generalisation of Ordinary Kriging (OK): OK treats the covariance structure as static and globally bounded by hard search neighbourhoods. Framing OK as a degenerate case of a continuous spatial operator allows spatial estimation to be viewed not as a discrete matrix inversion problem, but as an evaluation of an underlying continuous functional space.

Premise 2 - Weight convexity conservation: The linear redistribution rule for sample exclusion, $w_{i'} = w_i + p_i w_k$ subject to $\sum p_i = 1$, strictly preserves the non-bias constraint $\sum w_i = 1$. This confirms that sample exclusion or down-weighting operates as a continuous mapping within the convex hull of valid spatial weights rather than requiring a total re-solvability step of the linear system $Kw = k$.

Premise 3 - Equivalence of updating dynamics: The equivalence between recursive, incremental weight updates (e.g., Kalman filter state estimation) and structural domain modification (Fuzzy Domain Kriging) demonstrates that spatial domain decay and temporal update bounds are dual representations of the same underlying projection operator.

The continuous limit, from discrete matrices to infinite fields

The proposition that discrete Kriging is a finite projection of an infinite-dimensional field aligns directly with functional analysis and Gaussian Process (GP) regression over continuous domain spaces.

In standard OK, the linear system is written as:

$$\begin{bmatrix} K & 1 \\ 1^T & 0 \end{bmatrix} \begin{bmatrix} w \\ \mu \end{bmatrix} = \begin{bmatrix} k \\ 1 \end{bmatrix}$$

When extending the discrete set of N points to an infinite spatial domain continuous operator Ω, the matrix inversion K^{-1} transitions into an integral operator kernel $K^{-1}(x, x')$.

- FDK as a continuous operator: Rather than discarding samples outside an arbitrary radius R, Fuzzy Domain Kriging includes all spatial points by modulating their support via continuous membership functions. This turns the discrete sampling mask into a smooth, spatially continuous field operator.
- The Global Model (G): $G(x)$ represents the infinite spatial correlation field across the entire domain Ω, defined by the global covariance topology and stationary kernel properties.

Local adaptation via operator decomposition $(G \cdot V)$

Expressing the spatial field as a non-stationary spatial operator S(x)=G(x)·V(x) formalises how local variation modulates global spatial dependence:

$$\hat{Z}(x_0) = \int_{\Omega} V(x, x_0) G(x, x_0) Z(x) dx$$

- Global Field $G(x, x_0)$: Provides the stationary background spatial autocorrelation function across the infinite domain.
- Local adaptation vector/field $V(x, x_0)$: Acts as a spatially variable decay, anisotropy, or membership function (e.g., fuzzy membership or local variance scaling).
- OK as a binary specialisation: Standard Ordinary Kriging represents the degenerate case where $V(x, x_0)$ is a step function (hard indicator boundary):

$$V_{OK}(x, x_0) = \begin{cases} 1 \text{ if } \ || \ x - x_0 \ || \leq R \\ 0 \text{ if } \ || \ x - x_0 \ || > R \end{cases}$$

By relaxing $V(x, x_0)$ to a continuous mapping $V : \Omega \to [0, 1]$, FDK bridges discrete neighbourhood estimation and global continuous field modelling, eliminating spatial boundary discontinuities.

Domain scaling and operator non-commutativity

When introducing domain decay, scaling, or fuzzy membership matrices D, the positioning of the linear transformation matrix or differential spatial operator relative to the inverse covariance matrix K^{-1} dictates whether modification applies to data space, covariance space, or weight space.

Post-weighting vs spatial pre-filtering

- $(kK^{-1})D$ (Post-weighting transformation): Modulates the solved spatial weights $w^T = kK^{-1}$ after global inversion. This scales the relative influence of individual sample weights dynamically at the estimation location.
- $D(kK^{-1})$ (Covariance mapping / dual operator): When D acts on the left, it transforms the entire continuous weighting functional. Because matrix and operator multiplication are non-commutative ($D(kK^{-1}) \neq (kK^{-1})D$), pre-multiplying maps the input cross-covariance structure into a transformed manifold prior to inversion, whereas post-multiplying alters the target weight distribution post-inversion.

Internal covariance modulation vs external covariance scaling

- $k(DK)^{-1}$ (Inversion warp): The matrix D is embedded inside the inversion. This scales the spatial variance manifold directly before inversion, effectively acting as an inverse metric tensor. Rather than scaling sample points independently, DK^{-1} represents a spatially non-stationary covariance distortion. This formulation acts as a regularised pseudo-inverse, where D injects local anisotropy, fuzzy domain decay, or metric space warps directly into the precision matrix (the spatial topology of the inverse problem).
- $k(DK^{-1})$ (Cross-covariance weight warper): The global precision matrix K^-1 is already computed, and D acts as a local operator that scales or rotates the row contributions of the precision matrix prior to inner-product multiplication with the cross-covariance vector k.

Additive noise, regularisation, and spatial offsets

The structural placement of an additive operator D distinguishes structural covariance regularisation from post-hoc spatial drift corrections.

- $k(K^{-1} + D)$ (Internal matrix regularisation): $D \in \mathbb{R}^{N\times N}$ is added directly to the precision matrix K^{-1}. Operates as a spatial metric regulariser or generalised

Tikhonov perturbation. It modifies the global connectivity between all sample pairs simultaneously. Adding D inside the parentheses acts as a localised spatial variance sink (e.g., accounting for non-stationary sensor noise, micro-scale variance, or variable nugget effect across sub-domains).

- $((kK^{-1}) + D)$ (Additive field offset) $D \in \mathbb{R}^{1\times N}$ (or an additive scalar field offset) is appended to the computed weights $w^T = kK^{-1}$. Represents an explicit spatial bias vector or baseline domain offset. It directly shifts the linear combination of weights without altering the underlying precision matrix structure, functioning as a residual drift model or exogenous forcing term.

Summary

Operator Variant	Target Subspace	Structural Role	Physical Analogy
$(kK^{-1})D$	Weight Space (w)	Post-hoc weight scaling / continuous membership decay Local	fuzzy domain scaling
$k(DK^{-1})$	Cross-Precision Space (K^{-1})	Direct row-scaling of pre-inverted precision matrix	Dynamic sample-to-precision weight projection
$k(DK)^{-1}$	Covariance Manifold (K)	Inverted domain coordinate transform / metric tensor warping	Non-stationary spatial manifold distortion
$k(K^{-1} + D)$	Covariance System	Additive precision regularisation / spatial variance sinks	Variable nugget and localised uncertainty
$(kK^{-1}) + D$	Output Weight Vector	Direct linear weight offset / trend shift	Exogenous drift and spatial bias

Proof

Ordinary Kriging baseline

In standard Ordinary Kriging, the augmented linear system is defined as:

$$\begin{bmatrix} K & 1 \\ 1^T & 0 \end{bmatrix} = \begin{bmatrix} k_0 \\ 1 \end{bmatrix} \Rightarrow w_{aug} = K_{aug}^{-1} k_{aug}$$

Where K is the $N \times N$ sample covariance matrix, k is the $N \times 1$ target covariance vector, w is the weight vector, and μ is the Lagrange multiplier enforcing $\sum w_i = 1$.

Proof for Variant A, multiplicative covariance scaling ($w = k(D \cdot K)^{-1}$)

Let $f = 1 - u$ be the out-of-domain complement vector. Define the interaction operator matrix D such that off-diagonal entries are scaled by cross-boundary uncertainty while diagonals remain $D_{ii} = 1$ to preserve self-variance:

$$D = I + (d_1 \cdot d_2) \odot (1 - I)$$

Where d_1, d_2 scale cross-covariance elements K_{ij} proportionally to $u_i u_j$.

Boundary limits and behaviour:

1. Full membership ($u_i = 1, \forall i$):

$$f_i = 0 \Rightarrow D = I \; (IdentityMatrix)$$

$$w = k(I \cdot K)^{-1} = kK^{-1} \; (Exact\ Ordinary\ Kriging)$$

2. Zero membership ($u_j = 0$):

Row j and Column j of $(D \cdot K)$ have off-diagonal entries driven to 0. Sample j becomes spatially decoupled from all other neighbourhood points, reducing its cross-covariance contribution $k_j \to 0$ and driving $w_j \to 0$.

Proof for Variant B, additive variance penalty ($w = k(K + R)^{-1}$)

Define $R(u)$ as a diagonal penalty matrix with elements:

$$R_{ii}(u_i) = \left(\frac{1 - u_i}{u_i}\right) \cdot \sigma^2_{sill}$$

Boundary limits and behaviour:

1. Full membership ($u_i = 1$):

$$R_{ii}(1) = (\frac{1-1}{1}) \cdot \sigma^2_{sill} = 0 \Rightarrow R(M) = 0_{N \times N}$$

$$w = k(K + 0)^{-1} \; (Exact\ Ordinary\ Kriging)$$

2. Zero membership ($u_j = 0$ for sample j):

$$R_{jj} \to \infty$$

Isolating row j from the linear system equation $(K + R)w + 1\mu = k$:

$$\sum_{m=1}^{N} K_{jm} w_j + R_{jj} w_j + \mu = k_j \Rightarrow w_j = \frac{k_j - \mu - \sum_{m \neq j} K_{jm} w_m}{K_{jj} + R_{jj}}$$

Taking the limit as $R_{jj} \to \infty$:

$$\lim_{R_{jj} \to \infty} w_j = 0$$

The weight for sample j is strictly zeroed out, and the remaining weights $w_{m \neq j}$ re-adjust via the Lagrange multiplier to satisfy $\sum w_m = 1$.

Proof for Variant C, post-projection weight scaling and equivalence

We seek a diagonal transformation matrix $M = diag(m_1, m_2, ..., m_N)$ such that:

$$w_C = w_{fixed} M = kK^{-1} M$$

To solve for M in terms of Variants A and B, set the weight vectors equal:

$$kK^{-1} M = k(K + R)^{-1}$$

Right-multiplying both sides by the inverse terms yields:

$$M = K \cdot (K + R)^{-1}$$

Using the Woodbury Matrix Identity on $(K + R)^{-1}$:

$$(K + R)^{-1} = K^{-1} - K^{-1}(R^{-1} + K^{-1})^{-1} K^{-1}$$

Substitute this back into the expression for M:

$$M = K\left[K^{-1} - K^{-1}(R^{-1} + K^{-1})^{-1} K^{-1}\right] = I - (R^{-1} + K^{-1})^{-1} K^{-1}$$

Unbiasedness normalisation and fast approximation for SGS

Because post-scaling breaks the sum-to-one constraint ($\sum (w_{fixed} M)_i \neq 1$), we normalise by the scalar sum:

$$w_{final} = \frac{w_{fixed} M}{1^T (w_{fixed} M)}$$

The SGS approximation: On a regular simulation grid, $w_{fixed} = kK^{-1}$ is calculated once for a fully populated master template. For any node visited along the random path, M is approximated directly as $M \approx diag(u_1, u_2, ..., u_N)$.

This eliminates matrix inversions during simulation:

$$w_{SGS} = \frac{w_{fixed} \odot u}{1^T (w_{fixed} \odot u)}$$

At $u_j = 1$: Sample retains its pre-computed weight w_j. At $u_j = 0$: Sample weight drops instantly to 0.

The source

The original workings in Microsoft Excel and prototype R code and toy example can be found in my mining-ds-toolkit GitHub repository. There is also newer R code and an implementation in Go.

The next challenge

While the linear algebra for point estimates is established, several key areas require further theoretical and practical exploration:

- Kriging Variance and uncertainty propagation:
  - Theoretical Alignment: In traditional hard-domain models, estimation variance drops artificially near wireframes due to local sample density, ignoring boundary placement error. Under FDK, because sample weights w are dampened by R or D near fuzzy contacts, the term $w^T k$ decreases, causing $\sigma^2_{FDK} = \sigma^2 - w^T k + \mu$ to increase naturally.
  - Open Challenge: Formalise how the decay rate of $u_i(d)$ can be parameterised to explicitly separate geometric boundary location uncertainty (where is the contact?) from geochemical contact fuzziness (how sharp is the grade transition?). Proof is needed to show whether σ^2_{FDK} provides an accurate, unbiased confidence interval for downstream resource classification (e.g., preventing misclassification of boundary blocks as Measured).
- Defining membership functions (u_i): Developing objective, reproducible methods to derive u_i (e.g., signed-distance field decays, logistic contact curves, or probability outputs from implicit machine learning models).
- Benchmarking and conditioning: Conducting empirical benchmarks against traditional hard/soft wireframe boundaries to evaluate:
  - Matrix condition numbers ($cond(K + R)$ vs $cond(D \cdot K)$) as $u_i \to 0$.
  - Reconciliation impact on block model tonnage/grade curves.
  - Real-world runtime speed-up of Variant C inside Sequential Gaussian Simulation pipelines.

By going through this exercise in modifying (extending) the kriging formulation, a very interesting premise is raised. That of a unified framework where kriging (Ordinary Kriging) is but a special case. The universal spatial estimator (USE) framework.

The Universal Spatial Estimator (USE) framework

Unified framework formulation

By treating standard Ordinary Kriging ($w^T = kK^{-1}$) as a baseline global continuous field, all local non-stationarities, domain decays, spatial offsets, and observation uncertainties aggregate into a single matrix equation.

Up until now the notation has been rather random. The matrix labels were selected simply to have different letters to represent the matrices. A more formal notation, standard notation is required to avoid notation collisions (such as A for area/design matrix, B for drift coefficients, or C for the spatial covariance matrix itself), the modifiers can be assigned letters that explicitly reflect their geometric or algebraic role using the Greek alphabet:

$$w_{USE}^T = \Xi + \Phi k \Psi (\Gamma K \Omega + \Lambda)^{-1} \Theta + \Delta$$

Or, if sticking to standard Latin typography for standard Go package API readability:

$$w_{USE}^T = U_0 + M_{in} k M_{out} (LKR + S)^{-1} W_{in} + U_1$$

Table 1: Modifier matrix breakdown and typographical mapping

Proposed Symbol	Functional Name	Matrix Role	Mathematical / Geostatistical Purpose	Identity State (Null)
L (or Γ)	Left-Covariance Warper	Matrix Pre-Multiplier ($\mathbb{R}^{N\times N}$)	Scales/warps spatial anisotropy prior to global covariance evaluation.	$I_{N\times N}$
R (or Ω)	Right-Covariance Warper	Matrix Post-Multiplier ($\mathbb{R}^{N\times N}$)	Spatial domain transform mapping output sample coordinate spaces.	$I_{N\times N}$
S (or Λ)	Spatial Variance Sink	Additive Noise/ Nugget ($\mathbb{R}^{N\times N}$)	Injects non-stationary nugget variances, sensor errors, or diagonal regularisers.	$0_{N\times N}$
W_{in} (or Θ)	Input Weight Modulator	Matrix Post-Multiplier ($\mathbb{R}^{N\times N}$)	FDK continuous membership scaling and domain decay ($\mu_i \in [0,1]$).	$I_{N\times N}$
M_{in} (or Φ)	Global Field Amplitude	Scalar Multiplier ($\mathbb{R}^{1\times 1}$)	Scales global field magnitude or acts as an overall gain parameter.	1
M_{out} (or Ψ)	Cross-Covariance Operator	Matrix Pre-Multiplier ($\mathbb{R}^{N\times N}$)	Warps point-to-estimation cross-covariance topology before precision-matrix projection ($k(DK^{-1})$)	$I_{1\times 1} = 1$
U_0 (or Ξ)	Baseline Prior Offset	Additive Vector Pre-Shift ($\mathbb{R}^{1\times N}$)	Global baseline structural weight shift (must satisfy $\sum U_{0,i} = 0$).	$0_{1\times N}$
U_1 (or Δ)	Residual Spatial Bias	Additive Vector Post-Shift ($\mathbb{R}^{1\times N}$)	Localised trend drift or ML residual bias correction ($\sum U_{1,i} = 0$).	$0_{1\times N}$

A core property of this framework is its algebraic fall-back. When any local modulation matrix is omitted or undefined, it defaults to its corresponding identity operator (I for multiplicative structures or 0 for additive structures).

Architectural advantages of this form:

- Explicit separation of linear operations: Multiplicative operators (L,R,W_{in}) model spatial non-linearity, directional stretching, and fuzzy continuous boundaries, while additive operators (S,U_0,U_1) process high-speed $O(N^2)$ noise and drift adjustments.
- Numerically stable Go implementation: Modifiers act purely as forward matrix multiplications or vector additions. The core system matrix K is inverted (or decomposed via Cholesky factor LL^T) once. Updating local estimations reduces to fastBLAS routines without any risk of singular matrix errors at runtime.

Where the constituent operators map across four functional layers:

- Global base field (kK^{-1}): The pre-computed continuous spatial covariance structure built once over the global domain Ω.
- Covariance-space modifiers (M_{out},L,R,S): Spatial variance sinks, variable measurement errors, or non-stationary noise metrics added directly to the precision operator.
- Weight/sample-space modifiers (M_{in},W_{in}): Local domain masks, anisotropy stretch tensors, or fuzzy continuous membership decay matrices modulating inputs (W_{in}) and outputs (M).
- Additive spatial trends (U_0,U_1): Global baseline shifts (U_0) and localised spatial bias offsets (U_1).

Computational paradigm

The single-build global pre-inversion formulation fundamentally alters the computational efficiency of spatial estimation across scale:

- Build once, evaluate everywhere: The expensive matrix inversion $(LKR + S)^{-1}$ or continuous operator factorisation is performed once globally (or updated via low-rank Woodbury updates if local S perturbations are sparse).
- Decoupled system execution: In large-scale simulations or multi-point spatial interpolation, the background global covariance tensor remains frozen in memory.
- Local adaptation via fast tensor products: Local domain variations (e.g., FDK variants, localised dynamic search windows, micro-climate barriers) are applied instantaneously at runtime by multiplying the cached global operator $(LKR + S)^{-1}$ by sparse local transformation matrices M_{in}, M_{out}, and W_{in}.

Integration with ML/AI trend models (hybrid estimation)

Refactoring geostatistics around this framework fits naturally with modern Machine Learning residual pipelines (e.g., Random Forest Kriging, Neural Spatial Process Models):

$$\hat{Z}(x_0) = \underbrace{f_{ML}(x_0)}_{Global\,Trend\,(AI/ML)} + \underbrace{w_{USE}^T(Z - f_{ML}(X))}_{Spatially\,Adapted\,Residual}$$

- ML Model (f_{ML}) captures high-dimensional, non-linear global trends across complex feature spaces (e.g., elevation, remote sensing data, meteorological covariates).
- USE Framework (w_{USE}) models the spatial autocorrelation of the residuals ($e = Z - f_{ML}$).
- Synergy with Modifiers: If ML uncertainty varies across regions, the local variance matrix R dynamically absorbs model variance sinks, down-weighting residuals in regions where the ML model is uncertain.

Simplification of complex geostatistical formulations

Many traditionally complex, standalone Kriging variants can now be refactored as lightweight config profiles over the same unified solver core:

- Universal / Trend Kriging: Replaced by assigning polynomial drift or ML trend outputs directly to the additive offsets (U_0,U_1).
- Heteroscedastic / Variable-Nugget Kriging: Replaced by passing point-specific error variances into the diagonal of S.
- Indicator and Fuzzy Kriging: Replaced by setting continuous membership function profiles in W.
- Moving Window Kriging: Replaced by applying a continuous spatial decay matrix W instead of truncating data matrices and re-inverting $(LKR + S)$ at every step.

This framework transforms what would usually be separate, complex Go packages into a single core engine with pluggable modifier interfaces.

Computational mechanics and structural insights

- Computational complexity difference: Matrix addition scales at $O(N^2)$ linear memory access overhead, whereas standard matrix multiplication scales at $O(N * 3)$ (or $O(N^{2.81})$ via Strassen's algorithm). For an $N \times N$ matrix, addition is roughly 100× to 1000× faster depending on CPU cache alignment and vectorisation.

- Preserving convexity ($\sum w_i = 1$): Multiplicative scaling allows row-stochastic or scalar normalization factors to naturally preserve or proportionally re-scale the sum of weights. Additive modifiers (U_0,U_1) shift values uniformly. To maintain $\sum w_i = 1$ under additive terms, any added offset vector $U_{1,i}$ must satisfy $\sum U_{1,i} = 0$ (a zero-sum mean-neutral perturbation).
- Inversion-free modifiers: Keeping all dynamic modifier matrices outside the inversion operator—or restricting modifications inside the inverse to additive noise matrices—avoids re-evaluating $O(N^3)$ inversions and prevents ill-conditioned matrix instabilities during runtime evaluations.

Software architecture

This USE framework promotes a solid separation of concerns and modularization for development of a high-performance spatial estimation library.

In a Go implementation, this modular structure cleanly separates into interfaces and lightweight struct composition:

```
// UniversalEstimator encapsulates the pre-inverted global model
// and accepts dynamic local modifiers at evaluation time.
type UniversalEstimator struct {
    GlobalKernel  CovarianceMatrix // Pre-computed K^-1 (built once)
    GlobalVector  func(target Point) Vector // Calculates k(x_0)
}

// LocalModifier applies runtime domain shifts, noise, or spatial masks.
type LocalModifier interface {
    ApplyV(kKInv Vector) Vector     // Weight/Domain scaling (FDK)
       ApplyR(K  CovarianceMatrix)  CovarianceMatrix  //  Variance/Nugget
injection
    ApplyD(kKInv Vector) Vector     // Spatial bias or drift offset
}
```

The design keeps the memory footprint low and execution speeds high.